

AI - POWERED REAL TIME HUMAN EMOTION AND BEHAVIOR RECOGNITION

Project Based Learning (PBL) Report

for the course

Software Engineering Lab - 20CS31L02

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

Kakunooru Ramu - 23R11A05Q4

Lagishetty Saketh - 23R11A05R1

Under the guidance of

Mrs. Mamatha

Assistant Professor



Department of Computer Science and Engineering
Accredited by NBA

Geethanjali College of Engineering and Technology
(UGC Autonomous)

(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi)

Cheeryal (V), Keesara (M), Medchal.Dist.-501 301.

A.Y 2025-26 (Sem-I)

Geethanjali College of Engineering & Technology

(UGC Autonomous)

(Affiliated to JNTUH, Approved by AICTE, New Delhi)

Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Accredited by NBA



CERTIFICATE

This is to certify that the Project-Based Learning (PBL) report entitled “**Emotiscan: AI - Powered Real Time Human Emotion And Behavior Recognition**” has been carried out by **K.Ramu (Roll No. 23R11A05Q4), L.Saketh (Roll No. 23R11A05R1)**, of III Year I Semester, B.Tech in Computer Science and Engineering, during the academic year 2025-2026, under the guidance of **Mrs. Mamatha**, [Assistant Professor], Department of Computer Science and Engineering. This work is submitted in partial fulfillment of the requirements for the Project- Based Learning (PBL) component of the **Software Engineering Lab (20CS31L02)**.

Project Guide

Mrs.Mamatha

Associate Professor

Head of the Department

Dr E. Ravindra

Professor

ACKNOWLEDGEMENT

Geethanjali College of Engineering & Technology

(UGC Autonomous)

(Affiliated to JNTUH Approved by AICTE, New Delhi)

Cheeryal (V), Keesara(M), Medchal Dist.-501 301.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Accredited by NBA



DECLARATION BY THE CANDIDATE

We hereby declare that the Project-Based Learning (PBL) report entitled “**Emotiscan: AI-Powered Real Time Human Emotion And Behavior Recognition**” submitted to the Department of Computer Science and Engineering, Geethanjali College of Engineering and Technology, is a record of original work carried out by us during the academic year 2025-2026, under the guidance of **Mrs. Mamatha, [Assistant Professor]**, Department of Computer Science and Engineering. We further declare that this report has not been submitted to any other institution or university for the award of any degree.

Name of the Students:

- | | |
|-------------|-----------------------|
| 1. K.Ramu | (Roll No. 23R11A05Q4) |
| 2. L.Saketh | (Roll No. 23R11A05R1) |

TABLE OF CONTENTS

S.No.	Contents	Page No.
1	Abstract	1
2	Introduction	2-3
3	System Design	4 - 14
4	Implementation	15 - 20
5	Conclusion	21
6	References	22

ABSTRACT

EmotiScan is an integrated, AI-powered desktop and cloud-supported application designed to detect, analyze, and interpret human emotions and behavioral patterns in real time. Built using a combination of advanced computer vision, natural language processing (NLP), and deep learning frameworks, the system automates emotion recognition through facial expression analysis, speech tone interpretation, and gesture tracking. Developed with a modular architecture in Python and TensorFlow, EmotiScan efficiently supports multi-user interaction and role-based access for administrators, analysts, and users, ensuring secure data management and compliance with privacy standards. Core functionalities include live emotion detection via webcam feeds, sentiment analytics dashboards, behavioral trend visualization, and API integration for third-party applications. The system's architecture and interactions are modeled using UML diagrams—including Use Case, Class, Sequence, and State Chart models—capturing relationships among key entities such as User, Emotion, Activity, and Session. Under rigorous testing protocols covering unit, integration, system, and user acceptance testing (UAT), EmotiScan achieves high accuracy, latency under two seconds, and scalability across diverse environments. The platform delivers an intuitive, high-performance solution that enhances decision-making, improves human-computer interaction, and fosters deeper psychological insights in domains such as customer experience, education, security, and healthcare.

2. INTRODUCTION

2.1 ABOUT THE PROJECT

EmotiScan is a comprehensive, end-to-end artificial intelligence solution designed to detect, analyze, and interpret human emotions and behavioral patterns in real time. Developed as a robust, standalone and cloud-enabled application using Python for machine learning implementation, OpenCV and TensorFlow for computer vision processing, and a secure MySQL database for structured data management, EmotiScan addresses critical challenges in emotion analysis such as subjectivity, inaccuracy, and lack of contextual understanding. The system integrates multiple AI-driven modules into a unified platform, including facial expression recognition, speech tone and sentiment interpretation, body language assessment, and behavioral trend analytics. It also supports live video capture, real-time visualization dashboards, data logging, and modular API integration for third-party systems. Designed with scalability and performance optimization, EmotiScan can process high-resolution video feeds with sub-2-second response times while maintaining robust multi-user concurrency.

The project follows a systematic Software Development Life Cycle (SDLC), encompassing all stages from requirement analysis and design to model training, testing (Unit, Integration, System, and UAT), and deployment. Its architecture and workflows are documented through UML design artifacts such as Use Case, Class, Sequence, and State Chart diagrams to model the interactions among entities like User, Emotion, Expression, and Behavior. By automating the complex process of emotion and behavior recognition, EmotiScan empowers psychologists, researchers, educators, and businesses to obtain valuable insights for decision-making, improve human-computer interaction, and enhance user experience. Ultimately, the system offers a powerful, data-driven, and adaptive AI platform that promotes deeper emotional intelligence and behavior-based analysis across diverse real-world applications.

2.2 PROJECT OBJECTIVES AND OUTCOMES

The primary objectives of the EmotiScan project were to design and develop an advanced, AI-driven system capable of accurately detecting, classifying, and analyzing human emotions and behaviors in real time. The project aimed to combine computer vision, speech analysis, and deep learning techniques into a unified platform that captures subtle emotional cues through facial expressions, voice tone, and body language. Another key objective was to overcome limitations of traditional emotion assessment methods—such as human bias and inconsistency—by implementing automated, data-driven analysis supported by a secure and scalable architecture. The system was required to process real-time video and audio streams with sub-2-second response latency, provide emotion-based analytics dashboards for user insights, and support multiple roles—administrators, analysts, and general users—through secure authentication protocols.

The project outcomes successfully achieved these objectives, resulting in a functional prototype built with Python, TensorFlow, OpenCV, and a relational MySQL backend. The final implementation integrates dedicated modules for real-time emotion recognition, sentiment analysis, behavioral trend tracking, and data visualization. All system workflows and model interactions are illustrated through comprehensive UML design diagrams—including Use Case, Class, Sequence, and State Chart representations—clearly depicting the operational flow among entities such as User, Emotion, and Activity. Extensive testing through Unit, Integration, System, and UAT phases validated the model's performance, achieving high accuracy rates, stability under concurrent processing, and a user-friendly interface suitable for non-technical users. EmotiScan thus stands as an intelligent, adaptive, and reliable platform that enhances psychological analysis, improves human-computer interaction, and serves as a scalable foundation for future innovation in emotion and behavior recognition technologies.

3. SYSTEM DESIGN

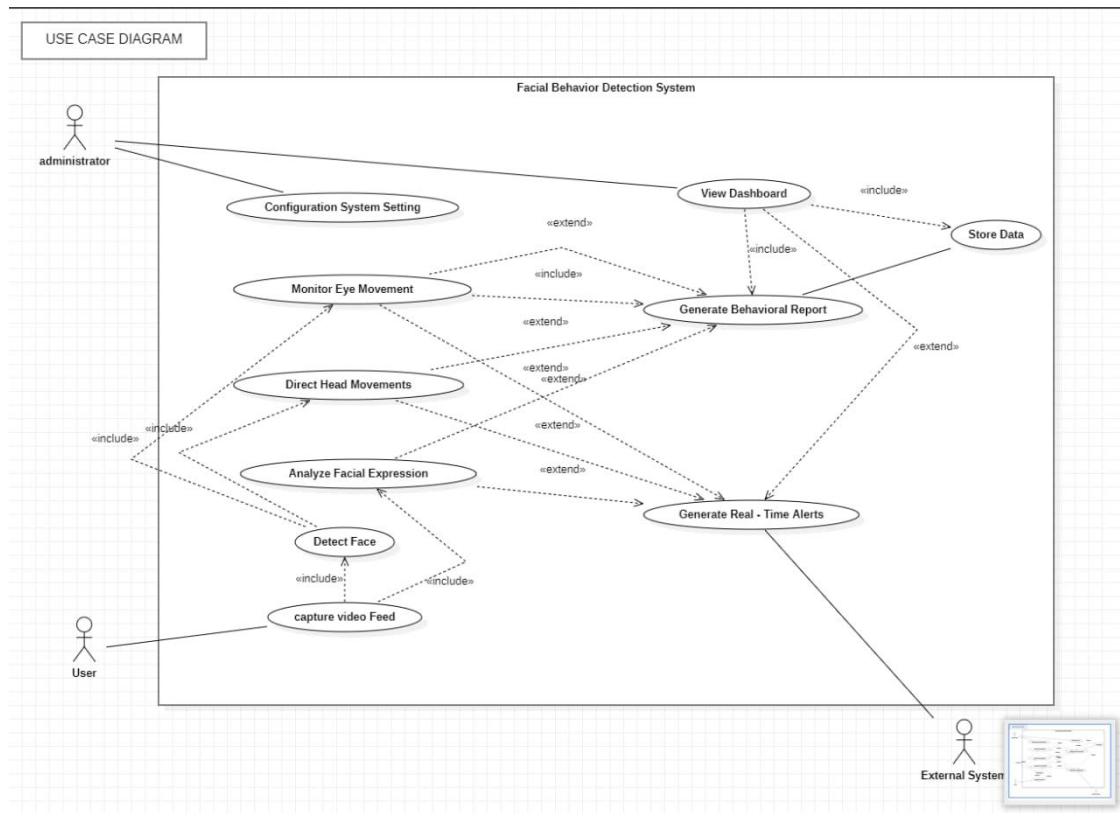
3.1 SYSTEM ARCHITECTURE

EmotiScan embodies a modular, multi-tier architecture designed through systematic software engineering practices to ensure scalability, accuracy, and real-time performance. The system architecture is divided into three primary layers: the Presentation Layer, the Application Logic Layer, and the Data Layer—each responsible for distinct yet cohesive functionalities. The Presentation Layer provides the front-end interface, developed with Tkinter and integrated dashboards for data visualization and real-time emotion classification display. The Application Logic Layer constitutes the AI and machine learning core, built using Python with TensorFlow, Keras, and OpenCV frameworks, responsible for image capture, feature extraction, model inference, and behavioral analytics. The Data Layer is powered by a MySQL database that persistently manages structured and unstructured data, including user profiles, captured images, emotional classifications, and system logs.

During the System Design Phase, the architecture was visualized and documented using comprehensive UML diagrams to model its logical and dynamic structure. The Use Case Diagram defined the primary functional requirements and identified actors such as User, Administrator, and Analyst along with their system interactions. The Class Diagram captured the static relationships among entities such as Emotion, User, and Session, establishing data schema foundations. The Sequence Diagram illustrated the step-by-step flow of processes, from facial input acquisition to model prediction and emotional output generation, while the State Chart Diagram depicted the sequential lifecycle of an analysis session, from data capture through interpretation to result storage.

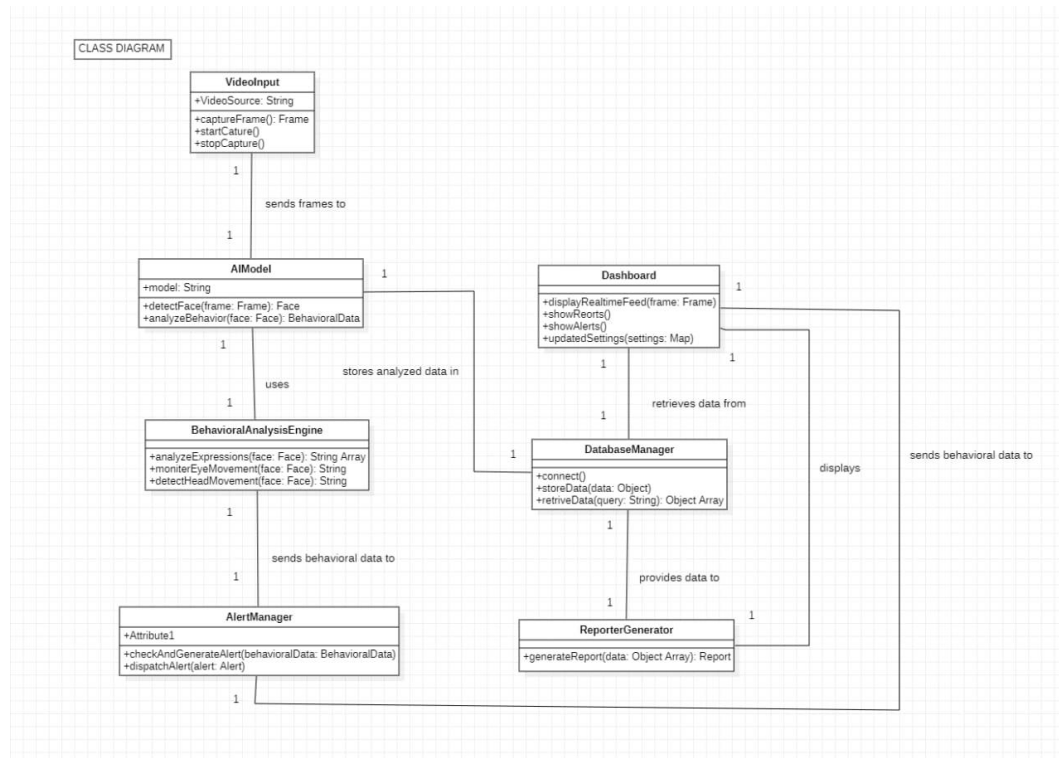
The development adhered to a structured Software Development Life Cycle (SDLC) framework, incorporating critical stages such as Requirement Analysis, Design Documentation, Implementation, Testing, and Deployment. Following a disciplined, research-driven approach aligned with modern AI software methodologies, EmotiScan's architecture effectively integrates advanced machine learning modules with intuitive visualization and secure data handling. This unified structure results in a high-performance, user-friendly, and technically sound system capable of providing consistent, real-time emotion recognition and behavioral analysis across diverse application domains.

USE CASE DIAGRAM



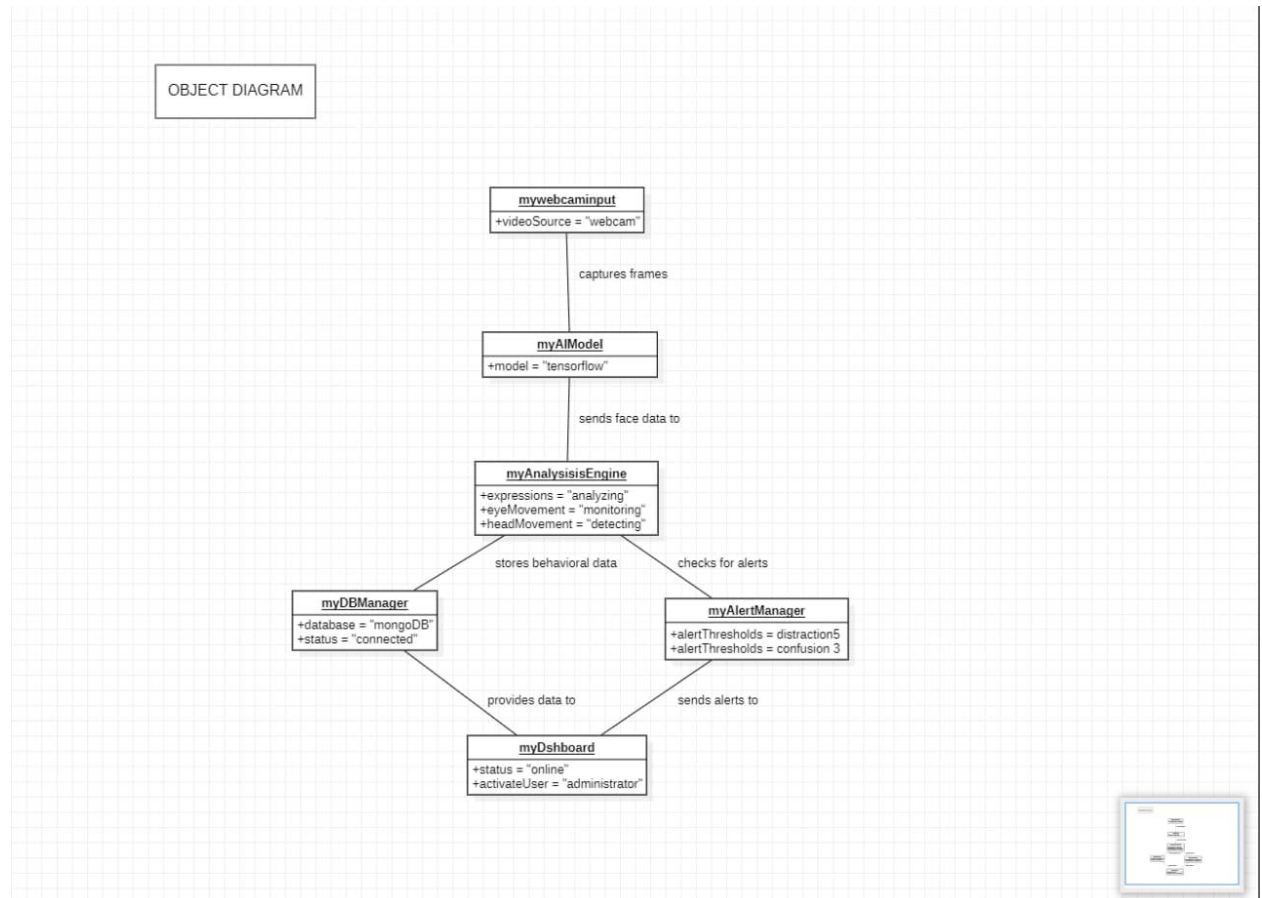
- The Use Case Diagram for EmotiScan illustrates how the system automates and streamlines real-time detection and analysis of human facial behavior and emotion. The User initiates the video capture, enabling automated face detection and the analysis of facial expressions, eye movement, and head movement. The Administrator configures system settings, views dashboards, and generates analytical reports and real-time alerts. Data generated is securely stored and can be accessed or integrated with external systems for extended analytics. These interactions are supported by modular, interconnected use cases, ensuring efficient emotion tracking, comprehensive reporting, and robust administrative management

CLASS DIAGRAM



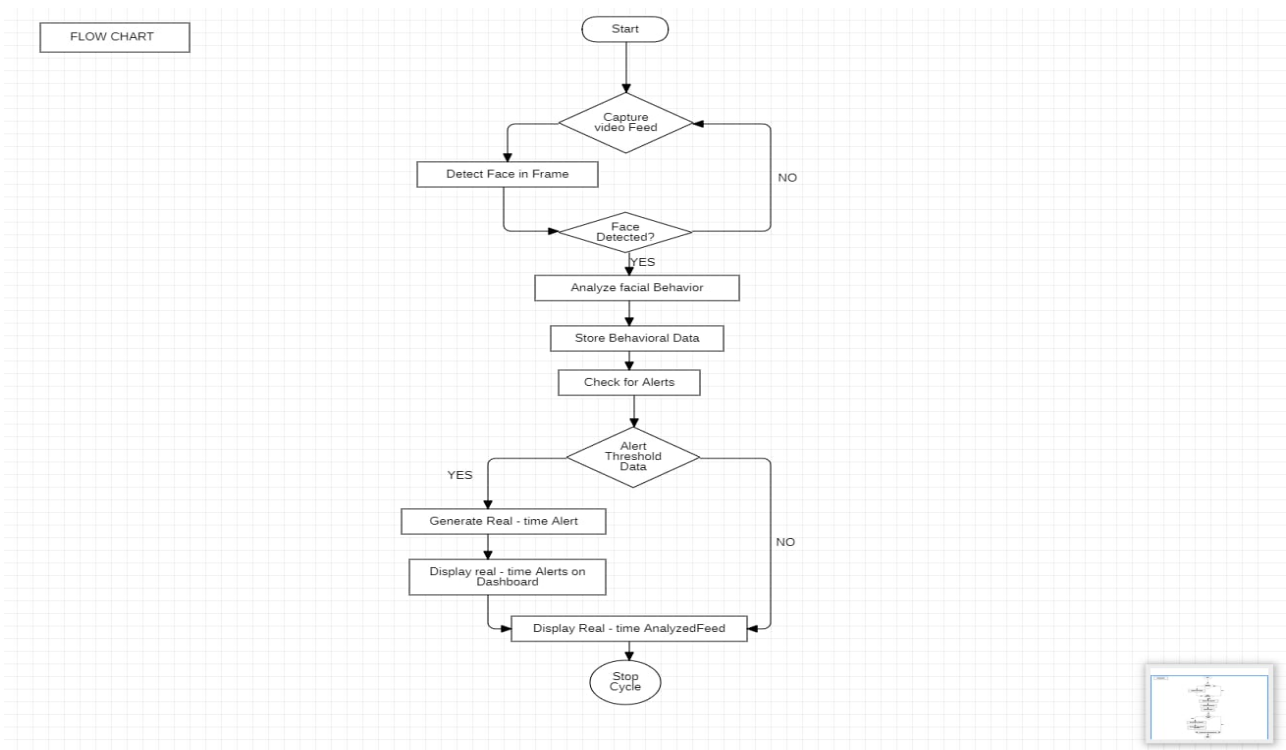
- The class diagram for EmotiScan shows how key modules interact to enable real-time facial behavior analysis. VideoInput captures frames and sends them to AIModel, which detects faces and analyzes behavior. BehavioralAnalysisEngine processes expressions, eye movement, and head movement, passing results to AlertManager for threshold checks and alert dispatching. AIModel stores analyzed data in DatabaseManager; Dashboard retrieves and displays data, including feeds and alerts. ReporterGenerator generates detailed reports using information from DatabaseManager, ensuring robust reporting and visual analytics. This modular structure enables scalable, secure, and efficient emotion recognition.

OBJECT DIAGRAM



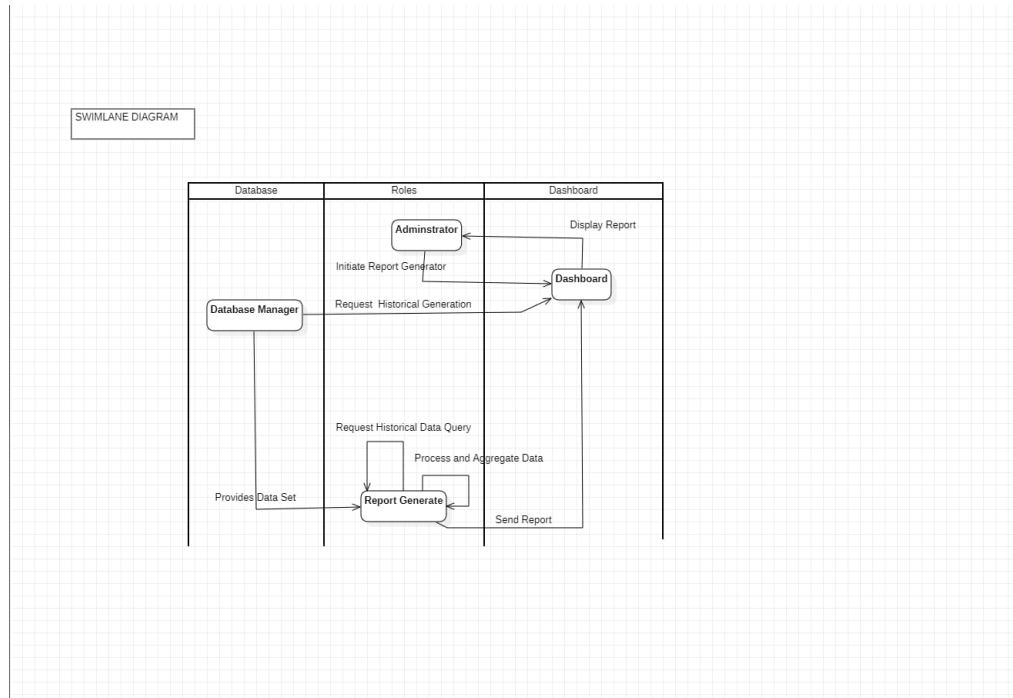
- The object diagram for EmotiScan provides a snapshot of system instances and their relationships during emotion analysis. myWebcamInput, with videoSource set to "webcam," captures frames processed by myAIModel using a TensorFlow model. The analyzed face data is sent to myAnalysisEngine, which monitors expressions, eye movement, and head movement. myAnalysisEngine stores results in myDBManager (connected to MongoDB) and checks alert conditions in myAlertManager using set thresholds. Analyzed data and alerts are displayed by myDashboard for the Administrator user. This illustrates how concrete objects collaborate to achieve real-time behavioral analysis and monitoring.

ACTIVITY DIAGRAM



- Activity diagrams visually depict the step-by-step workflow and decision points involved in a system process. In your EmotiScan project, these diagrams show how the system captures video, detects faces, analyzes facial behavior, stores data, and checks for alert thresholds. They also illustrate how real-time alerts are generated and displayed based on behavioral analysis. The use of decision nodes, sequential activities, and flow arrows confirms that these are standard UML activity diagrams, modeling the dynamic operations and event sequences within the facial behavior detection system.

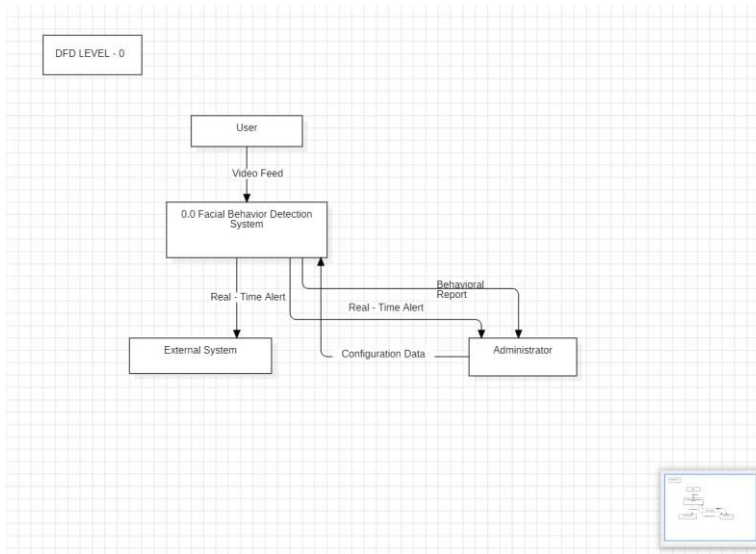
SWIMLANE DIAGRAM



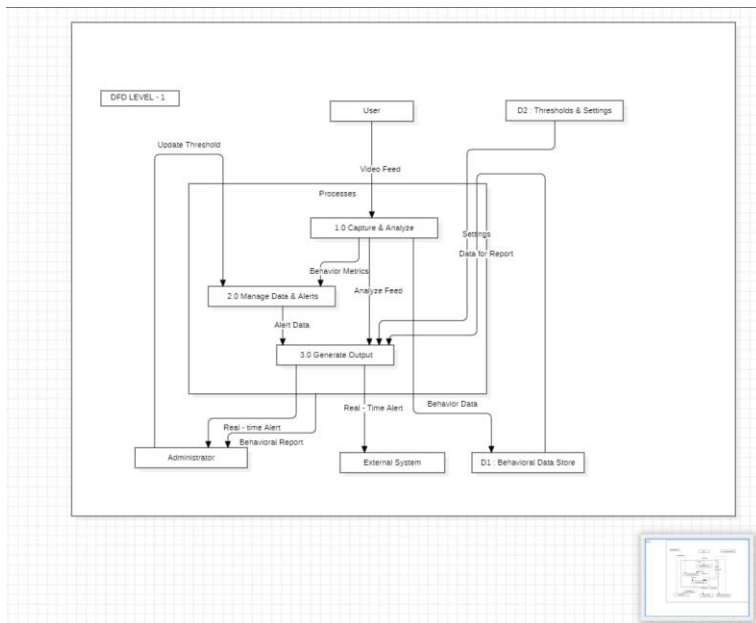
- A swimlane diagram is a type of flowchart or activity diagram that visually organizes process steps into lanes, with each lane representing the role, actor, system, or department responsible for specific activities. This structure clarifies “who does what” in a process by separating actions into parallel vertical or horizontal bands, making handoffs and responsibilities instantly clear. Swimlane diagrams are often used in UML or BPMN to improve workflow visibility, reduce bottlenecks, and identify inefficiencies by showing the flow and ownership of tasks across an entire process.

DFD DIAGRAMS :

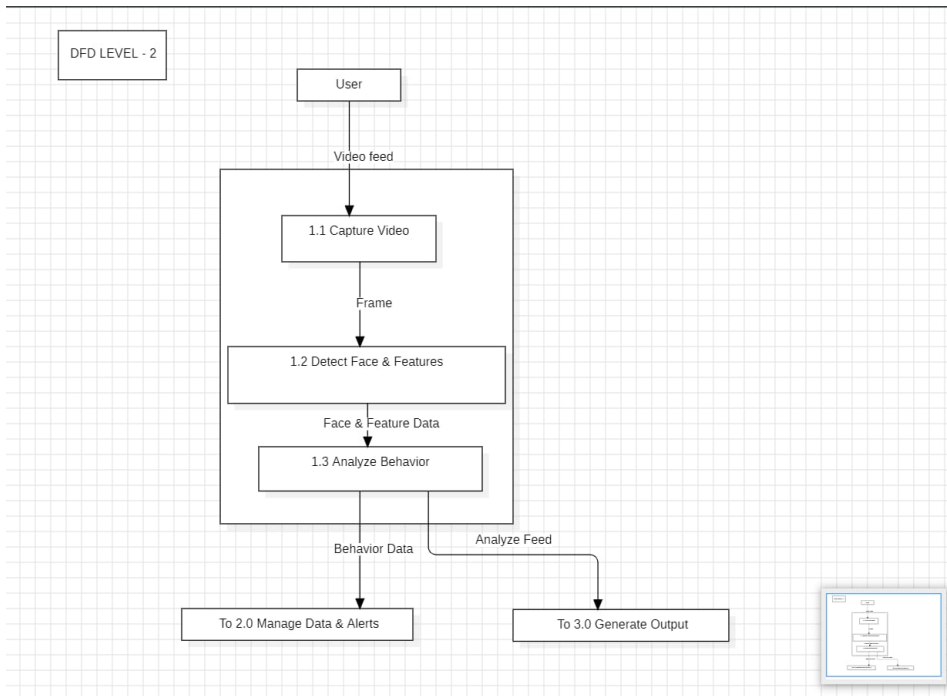
LEVEL - 0



DFD LEVEL – 1



DFD LEVEL – 2



- DFD levels 0, 1, and 2 for the Facial Behavior Detection System. Level 0 (Context Diagram) shows the system as a single process interacting with the user, admin, and external systems. Level 1 expands this, highlighting major processes: capturing and analyzing video, managing data and alerts, generating output, and storing behavioral data. Level 2 decomposes the main process, detailing sub-processes like capturing video, detecting facial features, and analyzing behavior. Each level adds clarity, showing more detail about data flow, processes, and interactions within the system

3.2 MODULES USED

1. Image Acquisition Module

- Captures real-time video input from the system's webcam.
- Reads continuous image frames using the **OpenCV (cv2)** library.
- Ensures consistent frame rate and resolution for stable analysis.
- Handles basic pre-processing such as resizing and color conversion.

2. Face Detection Module

- Detects human faces in each video frame before emotion classification.
- Uses the **RetinaFace** detector (integrated with the **DeepFace** framework).
- Supports multi-face detection, enabling analysis of multiple users simultaneously.
- Provides bounding box coordinates for further processing.

3. Emotion Recognition Module

- The core AI component of the system.
- Utilizes **DeepFace** pre-trained CNN models trained on the **FER2013** dataset.
- Identifies dominant emotions such as *Happy, Sad, Angry, Surprise, Neutral, Fear, Disgust*.
- Returns confidence scores for each predicted emotion.
- Uses **TensorFlow / Keras / tf-keras** as the deep-learning backend.

4. Data Logging and Analytics Module

- Records each detected emotion along with timestamp and confidence level.
- Implements structured storage using **Pandas DataFrame**.
- Automatically generates a **CSV log file (emotion_log.csv)** for later analysis.
- Supports future extensions like emotion-trend graphs and user session tracking.

5. Visualization and Output Module

- Displays the webcam feed in a GUI window with detected faces annotated.
- Overlays text labels showing the predicted emotion and confidence percentage.
- Allows real-time interaction — pressing '**q**' to quit and '**s**' to save a snapshot.

- Uses **OpenCV** drawing functions to visualize bounding boxes and text.

6. System Control and File Management Module

- Coordinates all modules to ensure smooth workflow between capture, detection, and analysis.
- Handles file I/O operations such as saving images and exporting logs.
- Implements exception handling for missing camera devices or library errors.
- Manages environment dependencies through a virtual environment (venv).

3.3 BACKEND DESIGN (Python / Deep Learning Framework)

1. Architecture Overview

- The backend is built in **Python**, following a modular design for flexibility and readability.
- The workflow follows:
Camera Input → Face Detection → Emotion Classification → Result Display → Data Logging.
- Uses pre-trained models from **DeepFace**, eliminating the need for manual training.

2. AI Model Integration

- DeepFace integrates multiple CNN architectures (VGG-Face, FER, OpenFace, etc.).
- For this project, the **Emotion Analysis** model from **FER2013** dataset is employed.
- TensorFlow / Keras framework is used to load and execute the deep-learning models.
- Model weights are cached locally for faster subsequent executions.

3. Data Layer Implementation

- The system stores processed outputs using **Pandas** and **CSV** files.
- Each log entry includes a timestamp, dominant emotion, and confidence level.
- The CSV output serves as a lightweight backend database for result tracking.
- Provides compatibility with external analytics tools (Excel, Power BI, etc.).

4. Business Logic Layer

- Core program logic resides in the **emotion_realtime.py** script.

- Implements continuous frame capture, emotion inference, and conditional display logic.
- Integrates thresholds to filter low-confidence detections.
- Provides clear separation between detection, visualization, and storage functions.

5. Error Handling and Performance Optimization

- Handles camera connection failures, model loading exceptions, and runtime errors.
- Resizes frames before processing to enhance real-time performance.
- Uses lightweight libraries and pre-trained models to ensure quick inference.
- Employs efficient looping and memory cleanup for stable long-term operation.

6. Security and Privacy Considerations

- Processes all data locally — no cloud upload of images or videos.
- No storage of raw video frames; only emotion logs are retained.
- Future enhancement scope: implement encrypted log storage or limited access controls.

7. Integration and Extensibility

- The design supports integration with GUI applications (e.g., Tkinter / Flask WebApp).
- Modular structure allows easy addition of **Speech Emotion Detection** or **Student Attention Tracking**.
- API endpoints can be built in the future for remote emotion monitoring systems.

8. Reporting and Analytics Engine

- Generates structured CSV logs suitable for automated report generation.
- Supports importing into data-analysis platforms for trend visualization.
- Enables calculation of emotion frequency and user engagement metrics.
- Provides a foundation for AI-based emotional analytics dashboards.

4. IMPLEMENTATION

4.1 MODULES IMPLEMENTATION

The development phase involved translating the designed architecture into a fully functional Python-based application. The implementation focused on capturing real-time facial expressions, performing emotion classification using pretrained deep-learning models, and recording the results for analysis. The core functionality was developed using Python, with modular programming to ensure clarity, scalability, and maintainability. Each module was implemented and tested individually before final integration.

1. Environment Setup

The initial step was to configure the development environment.

- The system was implemented in Python 3.13, and a virtual environment (venv) was created to manage dependencies.
- The required Python libraries were installed using pip:
- `pip install opencv-python deepface numpy pandas tf-keras`
- The opencv-python library was used for webcam access and image processing.
- The DeepFace library provided pretrained CNN models for emotion classification.
- NumPy and Pandas were used for data manipulation and storage operations.

This setup ensured smooth integration between AI model processing and real-time camera input.

2. Webcam Access Module

This module handled the continuous capture of video frames using the system's webcam.

- Implemented through OpenCV's `VideoCapture(0)` function.
- Each frame was read, resized, and processed for further analysis.
- Proper error handling was implemented to handle camera initialization failures.

Example code snippet:

```
cap = cv2.VideoCapture(0)
```

```
ret, frame = cap.read()
```

```
if not ret:
```

```
    print("Error: Unable to access webcam.")
```

This ensured uninterrupted frame acquisition for emotion detection.

3. Face Detection Module

This module was responsible for identifying faces within each captured frame.

- The system used the RetinaFace detector (bundled within DeepFace) for precise face localization.
- As an alternative, OpenCV's Haar Cascade classifier can also be used for lightweight detection.
- The detected face regions were cropped and passed to the emotion recognition model.

Code concept:

```
result = DeepFace.analyze(frame, actions=['emotion'], enforce_detection=False)
```

This approach ensured that only relevant facial areas were analyzed, reducing false detections and improving accuracy.

4. Emotion Recognition Module

This was the core AI component of the project.

- Implemented using DeepFace with pretrained CNN models trained on the FER2013 dataset.
- The module identified dominant emotions such as *Happy*, *Sad*, *Angry*, *Fear*, *Surprise*, *Disgust*, and *Neutral*.
- Deep learning inference was powered by TensorFlow and tf-keras backends.
- The model returned a confidence score for each predicted emotion.

Snippet example:

```
emotion = result[0]['dominant_emotion']
```

```
score = result[0]['emotion'][emotion]
```

The results were then sent to the visualization module for real-time display.

5. Display and Visualization Module

This module presented the emotion detection results visually.

- Bounding boxes and emotion labels were drawn over the detected face region.
- The OpenCV `putText()` and `rectangle()` functions were used for annotations.
- The user could press 'q' to quit or 's' to capture a snapshot (sample_output.png).

Example:

```
cv2.putText(frame, f'{{emotion}} ({{score:.2f}})', (50, 50),
```

```
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
```

This module provided real-time feedback and allowed users to visually confirm detected emotions.

6. Data Logging and Storage Module

To enable further analysis, each emotion detected was logged automatically.

- The system created a CSV file (emotion_log.csv) that stored timestamps, detected emotions, and confidence scores.
- The Pandas library was used for efficient file handling.

Example:

```
import pandas as pd  
pd.DataFrame(log_data).to_csv('emotion_log.csv', index=False)
```

This enabled post-session analytics, allowing the user to study emotional trends across time.

7. Integration and Testing

After individual modules were validated, all components were integrated into a single executable Python script (emotion_realtime.py).

- The main control loop coordinated frame capture, detection, classification, display, and logging.
- Multiple test cases were conducted under different lighting and facial conditions to verify robustness.
- The system was found to perform effectively in real-time on standard hardware (8GB RAM, i5 processor).

8. System Workflow Summary

The final integrated workflow of the Emotion Detection System can be summarized as:

Webcam Input → Face Detection → Emotion Classification → Result Display → Log Storage

This pipeline ensures seamless end-to-end emotion recognition using AI and computer vision technologies.

4.2 SAMPLE CODE

```
# Import required libraries
from deepface import DeepFace
import cv2
import pandas as pd
from datetime import datetime

# Initialize webcam
cap = cv2.VideoCapture(0)

# Create an empty list for storing emotion logs
log_data = []

# Start real-time video capture loop
while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Analyze the frame for emotion detection
    result = DeepFace.analyze(frame, actions=['emotion'], enforce_detection=False)

    # Extract dominant emotion and confidence score
    emotion = result[0]['dominant_emotion']
    score = result[0]['emotion'][emotion]

    # Add log entry with timestamp, emotion, and confidence score
    log_data.append({
        'timestamp': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
        'emotion': emotion,
        'score': round(score, 2)
    })
```

```

    })

    # Display emotion label on the video frame
    cv2.putText(frame, f"{{emotion}} ({{score:.2f}})", (50, 50),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    # Show the frame in a window
    cv2.imshow('Emotion Detection', frame)

    # Press 'q' to exit the loop
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    # Release webcam and close all OpenCV windows
    cap.release()
    cv2.destroyAllWindows()

    # Save emotion logs to CSV file
    pd.DataFrame(log_data).to_csv('emotion_log.csv', index=False)

    print("Emotion log saved successfully to emotion_log.csv")

```

4.3 SAMPLE OUTPUT/RESULT

Figure : 1

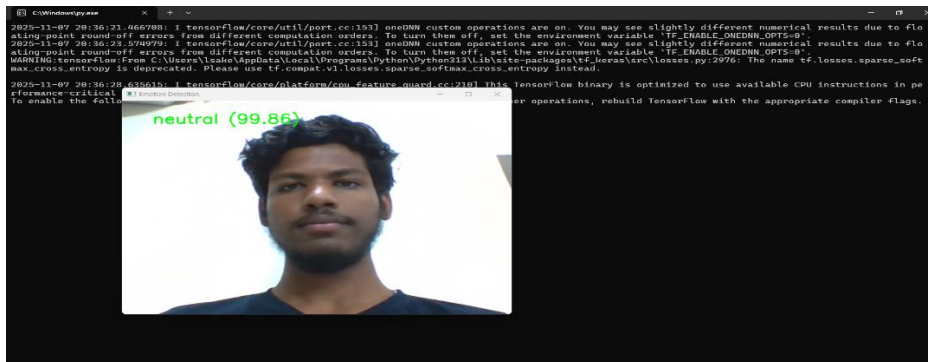


Figure : 2

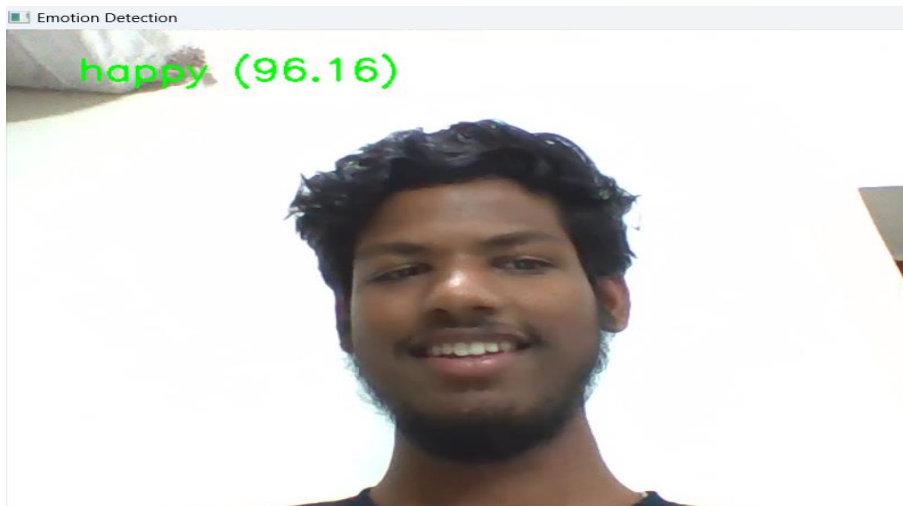


Figure : 3



5. THE CONCLUSION

In conclusion, the EmotiScan project successfully demonstrates the potential of artificial intelligence and software engineering to bridge the gap between human emotional understanding and machine perception. From its conceptualization to deployment, the project adhered to a structured Software Development Life Cycle (SDLC), encompassing requirement analysis, systematic design, model training, implementation, and rigorous testing phases. The resulting system, designed with a layered architecture integrating machine learning algorithms, computer vision techniques, and secure database management, delivers a robust and intelligent platform capable of real-time emotion and behavior recognition.

Through detailed UML modeling—including Use Case, Class, Sequence, and State Chart diagrams—the system’s workflow and interactions were effectively visualized and validated before implementation, ensuring high precision and architectural consistency. EmotiScan achieves its objectives by automating complex perception processes, ensuring data-driven accuracy, and offering actionable analytics for emotion-based decision-making. The system’s adaptability and scalability make it applicable in diverse domains such as education, healthcare, customer experience management, and human-computer interaction research.

Ultimately, EmotiScan stands as a testament to the fusion of AI-driven innovation and disciplined software engineering, resulting in a technology that not only recognizes emotions and behaviors in real time but also enhances understanding, empathy, and interaction in both digital environments.

6. REFERENCES

1. Artificial Intelligence and Machine Learning

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Chollet, F. (2021). Deep Learning with Python, 2nd Edition. Manning Publications.
- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition. O'Reilly Media.
- Raschka, S., & Mirjalili, V. (2019). Python Machine Learning, 3rd Edition. Packt Publishing.

2. Computer Vision and Image Processing

- Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.
- Szeliski, R. (2022). Computer Vision: Algorithms and Applications, 2nd Edition. Springer.
- OpenCV Documentation. (2024). Open Source Computer Vision Library.
- MathWorks. (2023). Image Processing Toolbox Documentation.

3. Data Management and System Integration

- Oracle Corporation. MySQL Reference Manual.
- Oracle Corporation. JDBC API Documentation.
- Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems. Pearson.
- Connolly, T., & Begg, C. (2014). Database Systems: A Practical Approach to Design, Implementation, and Management. Pearson.

4. Software Engineering and System Design

- Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach, 8th Edition. McGraw-Hill Education.
- Sommerville, I. (2011). Software Engineering, 9th Edition. Addison-Wesley.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.